

Controlled Measure-valued Martingales and Associated Machine Learning Methods

Joel Bassil



August 21, 2025

Motivation

- ▶ Consider an investor who wants to maximize their wealth by trading a risky asset.

Motivation

- ▶ Consider an investor who wants to maximize their wealth by trading a risky asset.
- ▶ Specifically, at time t , they place π_t units of currency of their total wealth X_t in the risky asset and seek to obtain the **value function**

$$v(x) = \sup_{\pi \in \mathcal{A}_{0,T}} \mathbb{E}[U(X_T^\pi)]$$

Motivation

- ▶ Consider an investor who wants to maximize their wealth by trading a risky asset.
- ▶ Specifically, at time t , they place π_t units of currency of their total wealth X_t in the risky asset and seek to obtain the **value function**

$$v(x) = \sup_{\pi \in \mathcal{A}_{0,T}} \mathbb{E}[U(X_T^\pi)]$$

- ▶ i.e the maximum **utility** over the set of admissible strategies.

Motivation

- ▶ Consider an investor who wants to maximize their wealth by trading a risky asset.
- ▶ Specifically, at time t , they place π_t units of currency of their total wealth X_t in the risky asset and seek to obtain the **value function**

$$v(x) = \sup_{\pi \in \mathcal{A}_{0,T}} \mathbb{E}[U(X_T^\pi)]$$

- ▶ i.e the maximum **utility** over the set of admissible strategies.
- ▶ The utility is essentially the investor's 'satisfaction' with their current wealth level.

Motivation

- ▶ Consider an investor who wants to maximize their wealth by trading a risky asset.
- ▶ Specifically, at time t , they place π_t units of currency of their total wealth X_t in the risky asset and seek to obtain the **value function**

$$v(x) = \sup_{\pi \in \mathcal{A}_{0,T}} \mathbb{E}[U(X_T^\pi)]$$

- ▶ i.e the maximum **utility** over the set of admissible strategies.
- ▶ The utility is essentially the investor's 'satisfaction' with their current wealth level.
- ▶ The investor would want to know both what their maximum utility could be, and the optimal control required to get there.

Motivation

- ▶ Consider an investor who wants to maximize their wealth by trading a risky asset.
- ▶ Specifically, at time t , they place π_t units of currency of their total wealth X_t in the risky asset and seek to obtain the **value function**

$$v(x) = \sup_{\pi \in \mathcal{A}_{0,T}} \mathbb{E}[U(X_T^\pi)]$$

- ▶ i.e the maximum **utility** over the set of admissible strategies.
- ▶ The utility is essentially the investor's 'satisfaction' with their current wealth level.
- ▶ The investor would want to know both what their maximum utility could be, and the optimal control required to get there.
- ▶ This optimization problem is known as the classical **Merton problem** (to be continued)

Main Objective

- ▶ This motivates the implementation of the **deep 2BSDE method**, a machine-learning based probabilistic numerical method used to solve fully-nonlinear PDEs and can be *linked* to optimal control problems.

Main Objective

- ▶ This motivates the implementation of the **deep 2BSDE method**, a machine-learning based probabilistic numerical method used to solve fully-nonlinear PDEs and can be *linked* to optimal control problems.
- ▶ Optimal control problems arise in multiple different areas of maths.

Main Objective

- ▶ This motivates the implementation of the **deep 2BSDE method**, a machine-learning based probabilistic numerical method used to solve fully-nonlinear PDEs and can be *linked* to optimal control problems.
- ▶ Optimal control problems arise in multiple different areas of maths.
- ▶ Consider a filtering problem, where the observer only has partial information that is revealed progressively over time through noisy feedback. This gives rise to an optimal control problem for a measure-valued process.

Main Objective

- ▶ This motivates the implementation of the **deep 2BSDE method**, a machine-learning based probabilistic numerical method used to solve fully-nonlinear PDEs and can be *linked* to optimal control problems.
- ▶ Optimal control problems arise in multiple different areas of maths.
- ▶ Consider a filtering problem, where the observer only has partial information that is revealed progressively over time through noisy feedback. This gives rise to an optimal control problem for a measure-valued process.
- ▶ This leads to the question of whether the deep 2BSDE method will work if the controlled process is measure-valued

Key Aims

- ▶ Understanding the construction of the **deep 2BSDE method**

Key Aims

- ▶ Understanding the construction of the **deep 2BSDE method**
- ▶ Implementation with a **measure-valued martingale process**

Key Aims

- ▶ Understanding the construction of the **deep 2BSDE method**
- ▶ Implementation with a **measure-valued martingale process**
- ▶ Investigation of results: **loss** and **optimal parameter choices**

2BSDE Method Construction

Problem: We wish to solve, for v , the fully nonlinear parabolic PDE

$$\begin{aligned}\frac{\partial v}{\partial t}(t, x) &= f(t, x, v(t, x), \nabla_x v(t, x), \text{Hess}_x v(t, x)) \\ v(T, x) &= g(x), \quad x \in \mathbb{R}^d\end{aligned}$$

2BSDE Method Construction

Problem: We wish to solve, for v , the fully nonlinear parabolic PDE

$$\begin{aligned}\frac{\partial v}{\partial t}(t, x) &= f(t, x, v(t, x), \nabla_x v(t, x), \text{Hess}_x v(t, x)) \\ v(T, x) &= g(x), \quad x \in \mathbb{R}^d\end{aligned}$$

Key Idea: Connect PDE to second-order BSDE system by defining the following:

$$\begin{aligned}dX_t &= b(u^{\text{aux}}, X_t)dt + \sigma(u^{\text{aux}}, X_t)dW_t, \quad X_0 = x_0 \\ Y_t &= v(t, X_t) \\ Z_t &= (\nabla_x v)(t, X_t) \\ \Gamma_t &= (\text{Hess}_x v)(t, X_t) \\ A_t &= \left(\frac{\partial}{\partial t} \nabla_x v \right) (t, X_t) + \frac{1}{2} \text{Tr}(\sigma(u^{\text{aux}}, X_t) \sigma^\top(u^{\text{aux}}, X_t) \Gamma_t)\end{aligned}$$

2BSDE Method Construction

Applying Itô's formula to Y_t and Z_t , we have:

► **Merged PDE-2BSDE system:**

$$\begin{aligned} Y_t &= g(X_T) - \int_t^T [f(s, X_s, Y_s, Z_s, \Gamma_s) \\ &\quad + \frac{1}{2} \text{Tr}(\sigma(u^{\text{aux}}, X_s) \sigma^\top(u^{\text{aux}}, X_s) \Gamma_s)] ds \\ &\quad - \int_t^T \langle Z_s, dX_s \rangle \\ Z_t &= Z_0 + \int_0^t A_s ds + \int_0^t \Gamma_s dX_s \end{aligned}$$

2BSDE Method Construction

Write the merged system in differential notation and then we have:

► **Forward discretization scheme (Euler-Maruyama):**

$$\begin{aligned}\mathcal{Y}_{n+1}^{\theta} &= \mathcal{Y}_n^{\theta} + \langle \mathcal{Z}_n^{\theta}, \mathcal{X}_{n+1} - \mathcal{X}_n \rangle \\ &\quad + \left[f(t_n, \mathcal{X}_n, \mathcal{Y}_n^{\theta}, \mathcal{Z}_n^{\theta}, \mathcal{G}_n^{\theta}(\mathcal{X}_n)) \right. \\ &\quad \left. + \frac{1}{2} \text{Tr}(\sigma(u^{\text{aux}}, \mathcal{X}_n) \sigma^{\top}(u^{\text{aux}}, \mathcal{X}_n) \mathcal{G}_n^{\theta}(\mathcal{X}_n)) \right] (t_{n+1} - t_n) \\ \mathcal{Z}_{n+1}^{\theta} &= \mathcal{Z}_n^{\theta} + \mathcal{A}_n^{\theta}(\mathcal{X}_n)(t_{n+1} - t_n) + \mathcal{G}_n^{\theta}(\mathcal{X}_n)(\mathcal{X}_{n+1} - \mathcal{X}_n) \\ \mathcal{X}_{n+1} &= \mathcal{X}_n + b(u^{\text{aux}}, \mathcal{X}_n)(t_{n+1} - t_n) + \sigma(u^{\text{aux}}, \mathcal{X}_n)(W_{t_{n+1}} - W_{t_n})\end{aligned}$$

Neural Network Approximation

- ▶ **Approximate unknown functions** using deep neural networks:
 - ▶ $\mathcal{G}_n^\theta$ approximates Hessian $(\text{Hess}_x v)(t_n, X_t)$
 - ▶ $\mathcal{A}_n^\theta$ approximates A_{t_n}

Neural Network Approximation

- ▶ **Approximate unknown functions** using deep neural networks:
 - ▶ $\mathcal{G}_n^\theta$ approximates Hessian $(\text{Hess}_x v)(t_n, X_t)$
 - ▶ $\mathcal{A}_n^\theta$ approximates A_{t_n}
- ▶ **Optimize parameters** by minimizing:

$$L(\theta) := \mathbb{E} \left[|\mathcal{Y}_N^\theta - g(X_T)|^2 \right]$$

- ▶ That is, we want the algorithm to learn the best starting point $\mathcal{Y}_0^\theta$ such that the final step gets as close to the terminal condition as possible.

Recap

- ▶ We have now constructed the deep 2BSDE method to solve a general PDE

Recap

- ▶ We have now constructed the deep 2BSDE method to solve a general PDE
- ▶ To implement it, we must define the PDE we wish to solve.

Recap

- ▶ We have now constructed the deep 2BSDE method to solve a general PDE
- ▶ To implement it, we must define the PDE we wish to solve.
- ▶ To solve a value function, like in the financial example, the PDE we solve should be a **Hamilton-Jacobi-Bellman (HJB) equation**, as this can be associated to a value function by the **dynamic programming principle (dpp)**.

HJB Implementation

- Suppose we define the **objective** as:

$$J(t, u; x_0) = \mathbb{E} \left[\int_t^T e^{-\beta(s-t)} k(u_s, X_s) ds + e^{-\beta(T-t)} g(X_T) \right]$$

where $k(u_s, X_s)$ is a cost function.

HJB Implementation

- ▶ Suppose we define the **objective** as:

$$J(t, u; x_0) = \mathbb{E} \left[\int_t^T e^{-\beta(s-t)} k(u_s, X_s) ds + e^{-\beta(T-t)} g(X_T) \right]$$

where $k(u_s, X_s)$ is a cost function.

- ▶ Then the **value** function is defined as:

$$v(t, x) := \inf_u J(t, u; x)$$

HJB Implementation

- Suppose we define the **objective** as:

$$J(t, u; x_0) = \mathbb{E} \left[\int_t^T e^{-\beta(s-t)} k(u_s, X_s) ds + e^{-\beta(T-t)} g(X_T) \right]$$

where $k(u_s, X_s)$ is a cost function.

- Then the **value** function is defined as:

$$v(t, x) := \inf_u J(t, u; x)$$

- By the **dynamic programming principle**, $v(t, x)$ satisfies the **HJB**:

$$\begin{aligned} \partial_t v - \beta v + \inf_u \left\{ k(u, x) + b(u, x) \cdot \partial_x v \right. \\ \left. + \frac{1}{2} \text{Tr}(\sigma(u, x) \sigma^\top(u, x) \partial_{xx}^2 v) \right\} = 0, \\ v(T, x) = g(x) \end{aligned}$$

HJB Implementation

- So we define f from the RHS of the PDE as:

$$f(\cdot) = \beta v - \inf_u \left\{ k(u, x) + b(u, x) \cdot \partial_x v + \frac{1}{2} \text{Tr}(\sigma(u, x) \sigma^\top(u, x) \partial_{xx}^2 v) \right\}$$

HJB Implementation

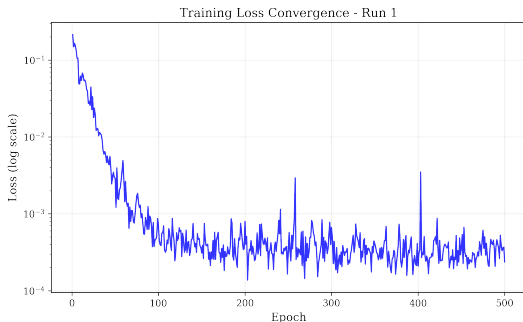
- So we define f from the RHS of the PDE as:

$$f(\cdot) = \beta v - \inf_u \left\{ k(u, x) + b(u, x) \cdot \partial_x v + \frac{1}{2} \text{Tr}(\sigma(u, x) \sigma^\top(u, x) \partial_{xx}^2 v) \right\}$$

- If we integrate this function into some code, the framework can solve any HJB equation by modifying only the cost, drift and diffusion functions.

Merton Problem - Continued

- Implementing this method with the financial example from earlier yields the following results:



- Loss converges appropriately

Merton Problem - Continued

Run 1 Results

Quantity	Value
Analytical solution	0.53839068
Deep2BSDE approximation	0.53861088
Absolute error	2.20e-04
Relative error (%)	0.0409



The algorithm estimates the solution to the value function up to an error of approximately 2×10^{-4} from the analytical solution.

Merton Problem - Continued

Run 1 Results

Quantity	Value
Analytical solution	0.53839068
Deep2BSDE approximation	0.53861088
Absolute error	2.20e-04
Relative error (%)	0.0409



The algorithm estimates the solution to the value function up to an error of approximately 2×10^{-4} from the analytical solution.

- ▶ One would then convert this solution back to wealth units, known as the **certainty equivalent** to understand the implications of the solution.

Main Idea - Measure-Valued Martingale Setup

- ▶ Consider a **measure-valued Martingale process** supported on $\{-1, 0, 1\}$ with initial position $\mu = \{\frac{1}{3}, \frac{1}{3}, \frac{1}{3}\}$:

$$d\xi_t = \sigma(u_t, \xi_t) dW_t$$

Main Idea - Measure-Valued Martingale Setup

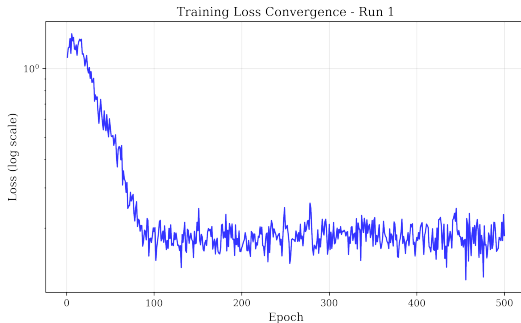
- ▶ Consider a **measure-valued Martingale process** supported on $\{-1, 0, 1\}$ with initial position $\mu = \{\frac{1}{3}, \frac{1}{3}, \frac{1}{3}\}$:

$$d\xi_t = \sigma(u_t, \xi_t) dW_t$$

- ▶ Here, σ is a diffusion function and we choose a cost function (both from Cox et al. 2024) such that the value function has the analytical solution $v(0, \mu) = 0$.

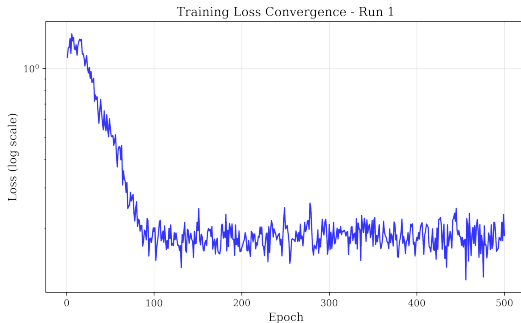
Implementation And Results

- Implementing the method with the above process and $T = 1$, cost function and constraints, we see the following results:



Implementation And Results

- Implementing the method with the above process and $T = 1$, cost function and constraints, we see the following results:



- The algorithm learns correctly and is able to approximate the terminal condition with a small loss.

Implementation And Results

- ▶ The solution is approximated correctly:

Run 1 Results

Quantity	Value
Analytical solution	0.00000000
Deep2BSDE approximation	-0.00190689
Absolute error	1.91e-03
Relative error (%)	inf

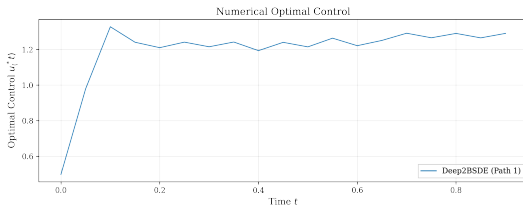
Implementation And Results

- ▶ The solution is approximated correctly:

Run 1 Results

Quantity	Value
Analytical solution	0.00000000
Deep2BSDE approximation	-0.00190689
Absolute error	1.91e-03
Relative error (%)	inf

- ▶ And the numerically approximated optimal control looks like:

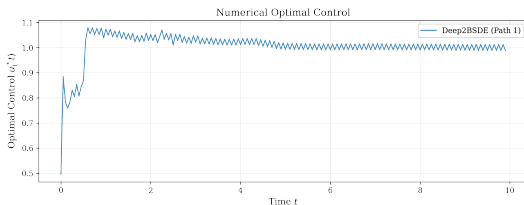


Implementation And Results

- ▶ In fact, the analytical solution for the optimal control is 1.

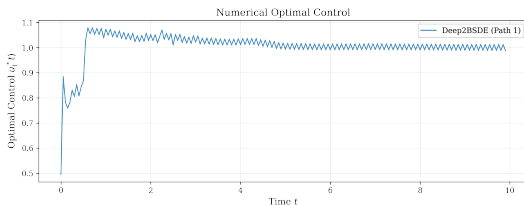
Implementation And Results

- ▶ In fact, the analytical solution for the optimal control is 1.
- ▶ If we run the algorithm over a longer time period and keep the discretization step-size the same, the optimal control looks like:



Implementation And Results

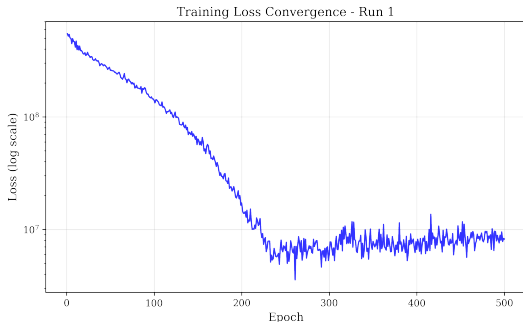
- ▶ In fact, the analytical solution for the optimal control is 1.
- ▶ If we run the algorithm over a longer time period and keep the discretization step-size the same, the optimal control looks like:



- ▶ So once the model has had sufficient time to learn, it finds the correct optimal control eventually

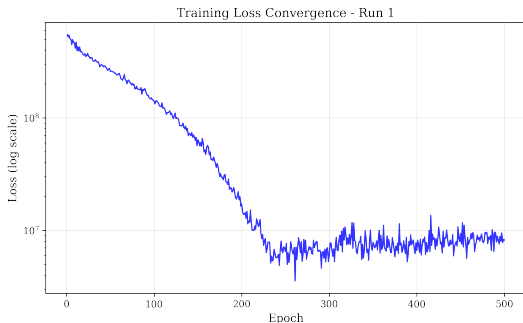
Loss Explosion

- ▶ However, the loss converges to approximately 10^7 , meaning the discretization is not learning appropriately to reach the correct terminal condition



Loss Explosion

- ▶ However, the loss converges to approximately 10^7 , meaning the discretization is not learning appropriately to reach the correct terminal condition



- ▶ This behavior prompts us to explore whether we can reduce this loss to increase accuracy of this method in the measure-valued case

Loss Explosion

- Recall the update equation for $\mathcal{Y}_{n+1}^\theta$. It turns out that, for large N , this recursion relation can be approximated by

$$\mathcal{Y}_N^\theta \approx \left(\mathcal{Y}_0^\theta + \frac{C_n N}{\beta T} + \frac{R_n}{\beta^2} \right) e^{\beta T} - \frac{C_n N}{\beta T} - \frac{R_n}{\beta^2}$$

where R_n, C_n are variables that are not constant, but stay very small over the course of the algorithm.

Loss Explosion

- Recall the update equation for $\mathcal{Y}_{n+1}^\theta$. It turns out that, for large N , this recursion relation can be approximated by

$$\mathcal{Y}_N^\theta \approx \left(\mathcal{Y}_0^\theta + \frac{C_n N}{\beta T} + \frac{R_n}{\beta^2} \right) e^{\beta T} - \frac{C_n N}{\beta T} - \frac{R_n}{\beta^2}$$

where R_n, C_n are variables that are not constant, but stay very small over the course of the algorithm.

- We want to find values for parameters N and β such that $\mathcal{Y}_N^\theta = \frac{1}{\beta}$, the terminal condition.

Loss Explosion

- ▶ That is, we want solutions in β of the function

$$\left(\gamma_0^\theta + \frac{C_n N}{\beta T} + \frac{R_n}{\beta^2} \right) e^{\beta T} - \frac{C_n N}{\beta T} - \frac{R_n}{\beta^2} - \frac{1}{\beta}$$

Loss Explosion

- ▶ That is, we want solutions in β of the function

$$\left(\mathcal{Y}_0^\theta + \frac{C_n N}{\beta T} + \frac{R_n}{\beta^2} \right) e^{\beta T} - \frac{C_n N}{\beta T} - \frac{R_n}{\beta^2} - \frac{1}{\beta}$$

- ▶ To see its behavior, we can plot β against the function value, and estimate $\mathcal{Y}_0^\theta$ as zero since it is initialized very close to zero.

Loss Explosion

- ▶ That is, we want solutions in β of the function

$$\left(\mathcal{Y}_0^\theta + \frac{C_n N}{\beta T} + \frac{R_n}{\beta^2} \right) e^{\beta T} - \frac{C_n N}{\beta T} - \frac{R_n}{\beta^2} - \frac{1}{\beta}$$

- ▶ To see its behavior, we can plot β against the function value, and estimate $\mathcal{Y}_0^\theta$ as zero since it is initialized very close to zero.
- ▶ Note that this requires estimating R_n and C_n . To do this, we can run the algorithm and print the values, however, to run the algorithm we need to choose a value of β first.

Loss Explosion

- ▶ That is, we want solutions in β of the function

$$\left(\mathcal{Y}_0^\theta + \frac{C_n N}{\beta T} + \frac{R_n}{\beta^2} \right) e^{\beta T} - \frac{C_n N}{\beta T} - \frac{R_n}{\beta^2} - \frac{1}{\beta}$$

- ▶ To see its behavior, we can plot β against the function value, and estimate $\mathcal{Y}_0^\theta$ as zero since it is initialized very close to zero.
- ▶ Note that this requires estimating R_n and C_n . To do this, we can run the algorithm and print the values, however, to run the algorithm we need to choose a value of β first.
- ▶ After running it several times with different values of β , values of $R_n = 0.5$ and $C_n = 0.1$ seem appropriate.

Loss Explosion

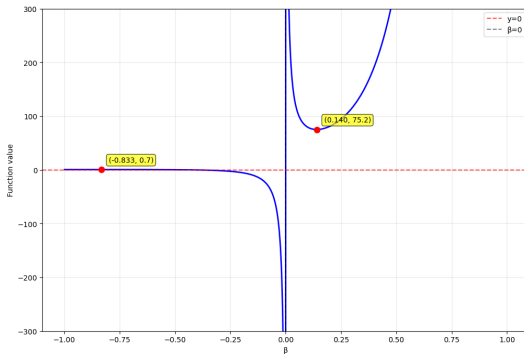
- ▶ That is, we want solutions in β of the function

$$\left(\mathcal{Y}_0^\theta + \frac{C_n N}{\beta T} + \frac{R_n}{\beta^2} \right) e^{\beta T} - \frac{C_n N}{\beta T} - \frac{R_n}{\beta^2} - \frac{1}{\beta}$$

- ▶ To see its behavior, we can plot β against the function value, and estimate $\mathcal{Y}_0^\theta$ as zero since it is initialized very close to zero.
- ▶ Note that this requires estimating R_n and C_n . To do this, we can run the algorithm and print the values, however, to run the algorithm we need to choose a value of β first.
- ▶ After running it several times with different values of β , values of $R_n = 0.5$ and $C_n = 0.1$ seem appropriate.
- ▶ In the future, finding bounds on these values could provide a more rigorous analysis

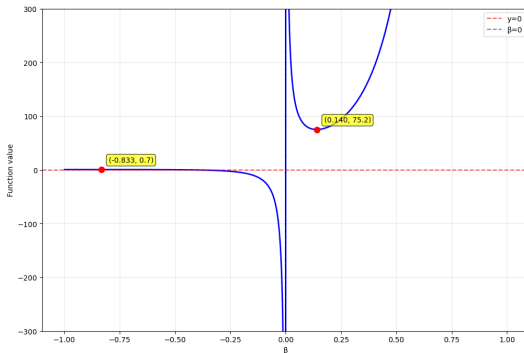
Loss Explosion

- The plot of β against the function value is as follows



Loss Explosion

- ▶ The plot of β against the function value is as follows



- ▶ This means that in this specific example, if our choice of R_n and C_n are accurate enough and β , N and T are not cooperative enough, then loss will never converge to a meaningful size.

Conclusions

- ▶ **Method Understanding:** Successfully constructed and implemented the deep 2BSDE method

Conclusions

- ▶ **Method Understanding:** Successfully constructed and implemented the deep 2BSDE method
- ▶ **Measure-Valued Extension:** Extended the method to measure-valued martingale processes with appropriate simplex constraints

Conclusions

- ▶ **Method Understanding:** Successfully constructed and implemented the deep 2BSDE method
- ▶ **Measure-Valued Extension:** Extended the method to measure-valued martingale processes with appropriate simplex constraints
- ▶ **Key Findings:**
 - ▶ Algorithm successfully approximates both value functions and optimal controls

Conclusions

- ▶ **Method Understanding:** Successfully constructed and implemented the deep 2BSDE method
- ▶ **Measure-Valued Extension:** Extended the method to measure-valued martingale processes with appropriate simplex constraints
- ▶ **Key Findings:**
 - ▶ Algorithm successfully approximates both value functions and optimal controls
 - ▶ Loss explosion in measure-valued case appears to be fundamental limitation for certain parameter ranges

Conclusions

- ▶ **Method Understanding:** Successfully constructed and implemented the deep 2BSDE method
- ▶ **Measure-Valued Extension:** Extended the method to measure-valued martingale processes with appropriate simplex constraints
- ▶ **Key Findings:**
 - ▶ Algorithm successfully approximates both value functions and optimal controls
 - ▶ Loss explosion in measure-valued case appears to be fundamental limitation for certain parameter ranges
- ▶ **Ideas For Future Work:** Investigate alternative discretization schemes, develop rigorous bounds on R_n and C_n parameters, and explore applications to higher-dimensional measure-valued problems

Thank You For Listening

Any Questions?

Early Optimal Control

- ▶ In both the Merton Problem example and the measure-valued case, the estimated optimal control performed poorly in early time steps.

Early Optimal Control

- ▶ In both the Merton Problem example and the measure-valued case, the estimated optimal control performed poorly in early time steps.
- ▶ We can calculate the optimal control at each time step as

$$u_t = \frac{B_t - A_t^2}{B_t - A_t^2 - \frac{1}{\beta} (B_t - A_t^2)^2 + \frac{T_t}{2}}$$

where A_t , B_t and T_t depend on the process ξ_t and the Hessian matrix Γ .

Early Optimal Control

- ▶ In both the Merton Problem example and the measure-valued case, the estimated optimal control performed poorly in early time steps.
- ▶ We can calculate the optimal control at each time step as

$$u_t = \frac{B_t - A_t^2}{B_t - A_t^2 - \frac{1}{\beta} (B_t - A_t^2)^2 + \frac{T_t}{2}}$$

where A_t , B_t and T_t depend on the process ξ_t and the Hessian matrix Γ .

- ▶ Since the analytical solution is 1, as mentioned earlier, we would see that $-\frac{1}{\beta} (B_t - A_t^2)^2 + \frac{T_t}{2} \rightarrow 0$.

Early Optimal Control

- ▶ In both the Merton Problem example and the measure-valued case, the estimated optimal control performed poorly in early time steps.
- ▶ We can calculate the optimal control at each time step as

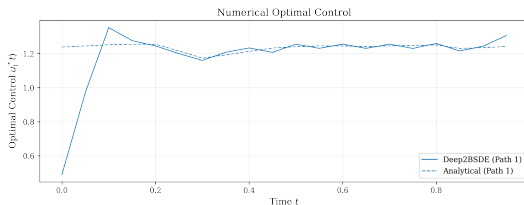
$$u_t = \frac{B_t - A_t^2}{B_t - A_t^2 - \frac{1}{\beta} (B_t - A_t^2)^2 + \frac{T_t}{2}}$$

where A_t , B_t and T_t depend on the process ξ_t and the Hessian matrix Γ .

- ▶ Since the analytical solution is 1, as mentioned earlier, we would see that $-\frac{1}{\beta} (B_t - A_t^2)^2 + \frac{T_t}{2} \rightarrow 0$.
- ▶ For this convergence to zero, the neural network approximating Γ needs an appropriate amount of time to learn and recover from poor random initialization.

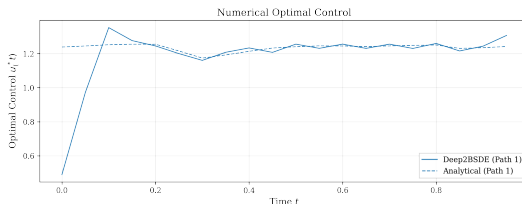
Early Optimal Control

- ▶ Plotting this optimal control against the numerical approximation looks like



Early Optimal Control

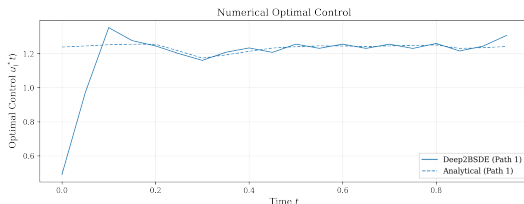
- ▶ Plotting this optimal control against the numerical approximation looks like



- ▶ Clearly, the early optimal control issue is with the optimizer and its initialization and learning rate.

Early Optimal Control

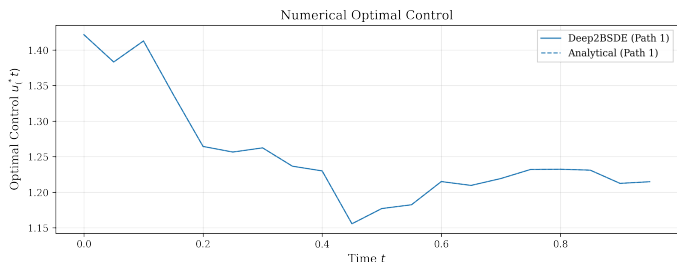
- ▶ Plotting this optimal control against the numerical approximation looks like



- ▶ Clearly, the early optimal control issue is with the optimizer and its initialization and learning rate.
- ▶ So far, we have used ADAM to find the optimal control, however, since there is only one variable to optimize over, an optimizer like L-BFGS may be more suitable.

Early Optimal Control

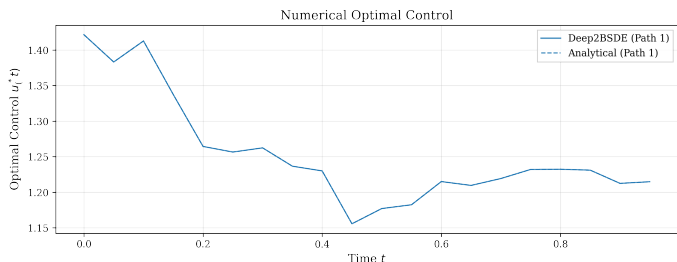
► Indeed,



using L-BFGS, there are no longer issues with the early (or otherwise) approximation of the optimal control.

Early Optimal Control

► Indeed,



using L-BFGS, there are no longer issues with the early (or otherwise) approximation of the optimal control.

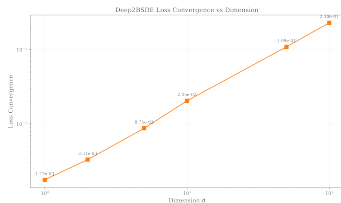
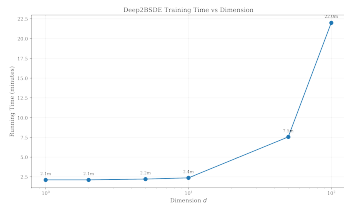
► Using this optimizer in the financial example may prove more useful than ADAM, which was used originally.

Classical Example Investigation

- ▶ One may wonder how modifying the dimension d may affect the performance of the algorithm.

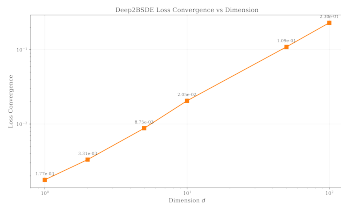
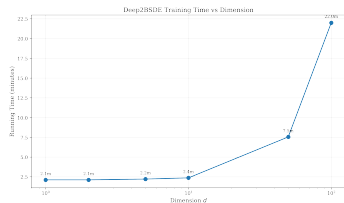
Classical Example Investigation

- ▶ One may wonder how modifying the dimension d may affect the performance of the algorithm.
- ▶ On a different example in the classical case, we see the following results



Classical Example Investigation

- ▶ One may wonder how modifying the dimension d may affect the performance of the algorithm.
- ▶ On a different example in the classical case, we see the following results



- ▶ As d increases, running time scales exponentially and loss scales linearly.

Simplex Constraints

- ▶ Since the process is measure-valued, we must impose some constraints

Simplex Constraints

- ▶ Since the process is measure-valued, we must impose some constraints
- ▶ Process must remain within simplex domain, i.e. $\sum_i (\xi_t)_i = 1$ and all components ≥ 0
 - ▶ We implement a Euclidean projection function to impose this

Simplex Constraints

- ▶ Since the process is measure-valued, we must impose some constraints
- ▶ Process must remain within simplex domain, i.e. $\sum_i (\xi_t)_i = 1$ and all components ≥ 0
 - ▶ We implement a Euclidean projection function to impose this
- ▶ $\mathcal{Z}$ must be in tangent space of simplex, therefore:
 - ▶ $\mathcal{A}_n^\theta \cdot \mathbf{1} = 0$

Simplex Constraints

- ▶ Since the process is measure-valued, we must impose some constraints
- ▶ Process must remain within simplex domain, i.e. $\sum_i (\xi_t)_i = 1$ and all components ≥ 0
 - ▶ We implement a Euclidean projection function to impose this
- ▶ $\mathcal{Z}$ must be in tangent space of simplex, therefore:
 - ▶ $\mathcal{A}_n^\theta \cdot \mathbf{1} = 0$
 - ▶ $\mathcal{G}_n^\theta$ symmetric with zero row sums